Signal Propagation in Single Cycle RV32I

Students: Kevin Gomez, Karellen Velazquez
Class: CIIC 4082
Professor: Prof. Juan Patarroyo

1. Introduction

This report documents the comprehensive testing and validation of a 32-bit RISC-V processor implemented in CircuitVerse. The objective is to verify correct operation of all major processor components and confirm proper instruction execution across the entire processor pipeline.

The project is organized into three phases:

1. **Part 1 - Instruction Memory Decoding**: Identification and analysis of the instruction set stored in the processor's instruction memory
2. **Part 2 - Individual Component Testing**: Systematic verification of each processor subsystem in isolation
3. **Part 3 - Processor Integration Testing**: End-to-end validation of the complete processor executing the full instruction sequence

Each phase includes detailed test protocols, measurements at critical signal points using the CircuitVerse Flag element, and verification that each component or instruction meets its functional specification.

The processor operates on 8-bit data but uses 32-bit instruction and program counter (PC) values, as specified in the reference design. Flags are strategically placed throughout the design to enable real-time signal observation and verification during simulation.

2. Processor Architecture Overview

The RISC-V processor implemented in this project follows a modular design with distinct functional blocks that interact through well-defined signal interfaces. Understanding the architecture is essential for developing effective test strategies and interpreting component behavior in context.

2.1. System Architecture

The processor is composed of the following primary subsystems:

TODO: add screenshots from CircuitVerse for each component

- **Instruction Memory (instr_mem)**: Holds the program instructions to be executed. Stores 8 words of 32-bit instruction data.

- **Register File (reg_file)**: Provides 8 registers for operand storage and intermediate computation results.
- **Control Unit (control_unit)**: Decodes instructions and generates control signals that coordinate subsystem operation.
- **Immediate Generator (imm_gen)**: Extracts and extends immediate values embedded within instructions.
- **Arithmetic Logic Unit (ALU)**: Performs arithmetic and logical operations on operands.
- **Data Memory (data_mem)**: Stores and retrieves program data during execution.

These components operate synchronously, with state changes occurring on clock edges. The processor employs synchronous reset, requiring the reset signal to be asserted high followed by a clock pulse to reinitialize the PC and memory elements.

2.2. Data Path and Signal Flow

Instructions are fetched sequentially from instruction memory based on the current PC value. The control unit decodes each instruction, extracting opcode, function codes, and register indices. Operands are retrieved from the register file, immediate values are generated as needed, and the ALU or data memory perform the specified operation. Results are written back to the register file, and the PC advances to the next instruction address.

Throughout execution, key signal points such as instruction values, operand data, ALU results, and memory access addresses are available for measurement and verification, enabling detailed observation of processor operation at any instruction cycle.

3. Part 1: Instruction Memory Decoding

The instruction memory block contains the program instructions that define the processor's behavior during validation.

3.1. Decoding Protocol

To decode the instruction memory contents, we accessed the `instr_mem` block within the CircuitVerse design and observed the binary instruction values stored at each memory address. Each 32-bit instruction follows the RISC-V instruction encoding format, with specific bit fields defining the operation, source registers, destination register, and immediate values.

The decoding process involved:

1. Identifying the opcode (bits [6:0]) to determine instruction type
2. Extracting the rd (destination register), rs1 (source 1), and rs2 (source 2) fields
3. Parsing function code fields (funct3, funct7) that refine the operation
4. Decoding immediate values for I-type, S-type, and B-type instructions
5. Translating the binary encoding to RISC-V assembly code

3.2. Documented Instructions

TODO: Insert decoded instructions and their assembly equivalents here.

The identified instructions form the test sequence executed during Part 3 processor validation.

4. Part 2: Individual Component Testing

Component-level testing validated that each processor subsystem functions correctly in isolation before integration into the complete data path. This phase ensures that issues are identified and resolved at the component level, simplifying debugging of whole-processor behavior.

The testing methodology for each component involved:

1. Developing a comprehensive test suite using the CircuitVerse TESTBENCH that exercises various operational modes and edge cases
2. Observing test results and ensuring they all pass
3. Documenting results to provide evidence of correct operation
4. Exporting test cases as csv files to save for future use and documentation

The following subsections document the testing of each major component:

4.1. Instruction Memory

The Instruction Memory is a read-only combinational memory that provides a 32-bit instruction word for a given 32-bit address. It is used by the instruction fetch stage to supply the next instruction based on the program counter.

4.1.1. Specifications

- **addr** (32-bit): Byte address input (word-aligned expected).
- **inst** (32-bit): Instruction word output stored at the given address.

The component performs a direct address-to-word lookup, addresses are expected to be aligned to 4-byte boundaries.

4.1.2. Testing Methodology

Verification supplies known 32-bit addresses and checks that the component returns the exact 32-bit instruction stored at each location. Tests exercise a contiguous region of addresses at the start of memory to confirm correct storage and address decoding.

4.1.3. Test Cases and Results

Input addr	Output inst
00000000000000000000000000000000 (0)	00000000000000000010000010000011

Input addr	Output inst
00000000000000000000000000000100 (4)	00000000001000001000000100110011
00000000000000000000000000001000 (8)	00000000001000000010000010100011
00000000000000000000000000001100 (12)	00000000001100011000010001100011
00000000000000000000000000010000 (16)	01000000011000111000001000110011
00000000000000000000000000010100 (20)	00000000001000001110001010110011
00000000000000000000000000011000 (24)	00000000000000000000000000010011
00000000000000000000000000011100 (28)	00000000000000000000000000010011

4.1.4. Verification

All supplied addresses returned the expected 32-bit instruction words. The Instruction Memory lookup and address decoding are verified for the tested address range.

4.2. Register File

TODO: fill in

4.2.1. Specifications

TODO: fill in

4.2.2. Testing Methodology

TODO: fill in

4.2.3. Verification

TODO: fill in

4.3. Control Unit

The Control Unit decodes the instruction opcode and function fields to generate the processor control signals. It translates `op`, `f3`, and `f7` inputs into branch, memory, ALU, and register control outputs used by the datapath.

4.3.1. Specifications

- **op** (7-bit): Opcode field from the instruction.
- **f3** (3-bit): funct3 field (function sub-code).
- **f7** (7-bit): funct7 field (function sub-code).
- **branch** (1-bit): Asserted for branch instructions.
- **mem_wr** (1-bit): Memory write enable for store instructions.

- **mem_rd** (1-bit): Memory read enable for load instructions.
- **mem_to_reg** (1-bit): Select memory data for writeback.
- **alu_ctrl** (3-bit): ALU operation code (mapped to ALU control inputs).
- **alu_src** (1-bit): Select immediate as ALU second operand.
- **reg_wr** (1-bit): Register file write enable.

Note: For load (`LW`), store (`SW`) and branch-equal (`BEQ`) instruction types, the `f3` and `f7` fields do not affect the control signals in this implementation; they are treated as don't-care bits marked with "X", this implies that injecting either "0" or "1" at the position yielded the exact same result.

4.3.2. Testing Methodology

The test suite provides opcode and function-field patterns and verifies the generated control signals. Test cases exercise:

1. **R-type variants**: Multiple `f3`/`f7` combinations that map to different `alu_ctrl` values and exercise register-write behavior, specifically for the ADD, SUB, AND, and OR instructions in that order.
2. **Load (LW)**: Confirms memory-read path and memory-to-register selection.
3. **Store (SW)**: Confirms memory-write enable and ALU source selection for addressing.
4. **Branch (BEQ)**: Verifies branch output assertion for branch-type opcodes.

Each category uses representative binary encodings from the CSV tests and checks the seven control outputs.

4.3.3. Test Cases and Results

R-Type Variants

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0110011	000	0000000	0	0	0	0	000	0	1
0110011	000	0100000	0	0	0	0	001	0	1
0110011	111	0000000	0	0	0	0	010	0	1
0110011	110	0000000	0	0	0	0	011	0	1

These R-type test cases validate decoding into register-write operations and correct `alu_ctrl` selection for different function fields.

Load (LW)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0000011	XXX	XXXXXXX	0	0	1	1	000	1	1

This test case confirms that a load instruction asserts `mem_rd`, selects memory data for writeback, and uses the immediate as the ALU source.

Store (SW)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0100011	XXX	XXXXXXX	0	1	0	0	000	1	0

The store test case verifies `mem_wr` assertion and that the ALU uses the immediate for address calculation; no register write is performed.

Branch (BEQ)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
1100011	XXX	XXXXXXX	1	0	0	0	001	0	0

The branch test case validates that the `branch` control is asserted for a branch-type opcode and that no memory or register writes occur.

4.3.4. Verification

All test cases produced the expected control outputs. The Control Unit decoding behavior matches the test encodings and the don't-care handling for `f3`/`f7` in LW, SW, and BEQ cases. The component is validated and ready for integration into the processor data path.

4.4. Immediate Generator

TODO: fill in

4.4.1. Specifications

TODO: fill in

4.4.2. Testing Methodology

TODO: fill in

4.4.3. Verification

TODO: fill in

4.5. ALU Component

The Arithmetic Logic Unit is a combinational circuit that performs binary arithmetic and logical operations. It serves as the primary computational engine for data manipulation throughout instruction execution.

4.5.1. Specifications

The ALU module operates on 8-bit operands and produces an 8-bit result along with status flags.

The output signals are:

- **result** (8-bit): The computed arithmetic or logical result
- **zero** (1-bit): A flag asserted high when the result equals zero

The ALU responds to a 3-bit control signal (`alu_ctrl`) that selects the operation, however the most significant bit is unused given only 4 operations, representable by 2 bits, are implemented. The most significant bit is hence marked as "X" meaning "don't care", this implies that injecting either "0" or "1" at the position yielded the exact same result. The supported operations are defined as follows:

Control Code	Operation	Description
X00	ADD	Binary addition of operands a and b
X01	SUB	Subtraction: a - b using two's complement
X10	AND	Bitwise AND of operands a and b
X11	OR	Bitwise OR of operands a and b

4.5.2. Testing Methodology

The ALU testing protocol exercises all supported operations across diverse operand patterns and edge cases. Six test categories ensure comprehensive coverage:

1. **Addition of Positive Numbers**: Validates binary addition with non-negative operands
2. **Addition of Negative Numbers**: Tests two's complement addition with negative operands
3. **Subtraction of Positive Numbers**: Verifies subtraction including cases producing negative results
4. **Subtraction of Negative Numbers**: Confirms signed subtraction with proper overflow behavior
5. **Bitwise AND Operation**: Validates bit-by-bit conjunction across operand patterns
6. **Bitwise OR Operation**: Confirms bit-by-bit disjunction including complementary patterns

Each category includes three test cases covering normal operation, zero results, edge values, and patterns exercising all bit positions.

4.5.3. Test Cases and Results

Addition of Positive Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
00000100 (4)	10000010 (130)	X00	10000110 (134)	0
00000101 (5)	00001001 (9)	X00	00001110 (14)	0
00000000 (0)	00000000 (0)	X00	00000000 (0)	1

Addition of positive operands produces correct sums with the zero flag asserted when both inputs

are zero.

Addition of Negative Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
11111100 (-4)	11111101 (-3)	X00	11111001 (-7)	0
11111111 (-1)	11111111 (-1)	X00	11111110 (-2)	0
10000000 (-128)	00000001 (1)	X00	10000001 (-127)	0

Two's complement negative operands are added correctly, with proper overflow handling within the 8-bit domain.

Subtraction of Positive Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
00001010 (10)	00000100 (4)	X01	00000110 (6)	0
00001111 (15)	00001111 (15)	X01	00000000 (0)	1
00000100 (4)	00001010 (10)	X01	11111010 (-6)	0

Subtraction of positive operands generates correct results, including cases where the result is negative.

Subtraction of Negative Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
11111100 (-4)	11111101 (-3)	X01	11111111 (-1)	0
10000000 (-128)	00000001 (1)	X01	01111111 (127)	0
11111111 (-1)	11111111 (-1)	X01	00000000 (0)	1

Signed subtraction using two's complement representation produces correct results including scenarios with sign bit inversion.

Bitwise AND Operation

Input a	Input b	alu_ctrl	Result	Zero Flag
11110000	00001111	X10	00000000	1
10101010	01010101	X10	00000000	1
11110011	11001100	X10	11000000	0

Bitwise AND correctly implements conjunction logic. Complementary bit patterns produce zero, while overlapping patterns generate appropriate non-zero results.

Bitwise OR Operation

Input a	Input b	alu_ctrl	Result	Zero Flag
11110000	00001111	X11	11111111	0
10101010	01010101	X11	11111111	0
00000001	00000010	X11	00000011	0

Bitwise OR correctly implements disjunction logic. Complementary patterns combine to produce all-ones results, and adjacent single-bit patterns merge correctly.

4.5.4. Verification

All ALU test cases execute successfully with results matching predicted values. The ALU correctly implements addition, subtraction, AND, and OR operations with proper two's complement handling and accurate zero flag generation. The component is validated and ready for integration into the processor data path.

4.6. Data Memory

TODO: fill in

4.6.1. Specifications

TODO: fill in

4.6.2. Testing Methodology

TODO: fill in

4.6.3. Verification

TODO: fill in

5. Part 3: Processor Integration and Validation

After verifying individual components, the complete processor must be validated executing the full instruction sequence. This integration phase confirms that components interact correctly and that the processor achieves the intended behavior when executing real programs.

5.1. Testing Protocol

Processor validation is performed using the CircuitVerse simulation environment shown below:

TODO: add screenshot of the full simulated circuit

TODO: add short intro

5.2. Signal Monitoring Points

Key signals to observe during execution include:

- Program Counter (PC): Current instruction address
- Current Instruction (Inst): The 32-bit instruction value fetched from instruction memory
- ALU Result: Output of the arithmetic logic unit
- Register File Write Enable and Write Data: Values being written to registers
- Data Memory Address, Write Enable, Read Enable Write Data, and Read Data: Memory access operations
- Zero Flag and Other Status Indicators: Condition flags from arithmetic operations

TODO: fill in (lo que esta puesto hasta ahora es porque eso imagino que va a ser pero no lo he actually hecho so no se)

5.3. Instruction Execution Verification

TODO: fill in

5.4. Processor Validation Results

TODO: fill in

6. Conclusion

Component-level testing confirmed the correct operation of individual subsystems, while integration testing verified proper interaction within the complete processor pipeline.

The structured approach to validation, testing components in isolation before system-wide integration, ensures that the processor architecture is sound and each element functions as designed. The comprehensive test protocols provide clear evidence of functional correctness and serve as a foundation for confident deployment of the processor design in subsequent applications.

All major processor subsystems have been verified to meet their functional specifications. The documented test results provide a complete record of validation activities and demonstrate the processor’s readiness for operation with the full instruction set.